

1. Recherche

1.1 Architekturtypen

Im Smart-Home-Bereich existieren cloud-zentrische Systeme wie Alexa oder Google Home, lokal betriebene Edge-first-Lösungen wie Home Assistant oder openHAB sowie hybride Architekturen. Der Trend im Jahr 2026 geht deutlich in Richtung hybrider, lokal-first Ansätze, da diese Datenschutz und geringe Latenz besser miteinander vereinen [floL26].

1.2 Plattformen

Tabelle 1.1: Übersicht relevanter Smart-Home-Plattformen

Plattform	Typ	Charakteristik	Relevanz 2026
Home Assistant	Lokal	Große Community und starke Matter-Unterstützung	Hoch
openHAB	Lokal	Hohe Flexibilität	Mittel
Apple Home	Hybrid	Gute Matter- und Thread-Unterstützung	Hoch
Google Home / Alexa	Cloud-dominant	Einfache Einrichtung und große Geräteauswahl	Mittel

1.3 Kommunikationsmodelle

Im Smart-Home-Umfeld dominieren zwei Kommunikationsmodelle: Publish-Subscribe und Request-Response. Besonders Publish-Subscribe, vor allem in Form von MQTT, hat sich durchgesetzt. Es ermöglicht eine starke Entkopplung von Sender und Empfänger sowie eine gute Skalierbarkeit. Gerade bei heterogenen und ressourcenbeschränkten Geräten erweist sich dieses Modell als vorteilhaft [OASI24].

1.4 Protokolle

Tabelle 1.2: Relevante Smart-Home-Protokolle im Überblick

Protokoll	Typ	Stärken und Einsatz 2026
Matter	Application Layer	Herstellerübergreifender Standard und aktuell stärkster Trend
Thread	Network Layer (Mesh)	Energieeffizient und selbstheilend, besonders für Batteriegeräte geeignet
MQTT	Application Layer	Leichtgewichtig und skalierbar, sehr weit verbreitet
Zigbee	Mesh	Große installierte Basis, wird zunehmend durch Matter und Thread ergänzt
Wi-Fi	Network	Hohe Bandbreite, vor allem für stromintensive Geräte

Matter ist kein reines Funkprotokoll, sondern ein Application-Layer-Standard, der über Thread, Wi-Fi oder Ethernet betrieben werden kann. Sein Hauptvorteil liegt in der verbesserten Interoperabilität zwischen Geräten unterschiedlicher Hersteller [Conn24].

Tabelle 1.3: Vergleich der Deployment-Modelle

Aspekt	Cloud-zentrisch	Edge-first	Hybrid
Latenz	Mittel bis hoch	Sehr niedrig	Niedrig
Datenschutz	Schwach	Sehr gut	Gut
Offline-Fähigkeit	Schlecht	Sehr gut	Gut
Skalierbarkeit	Sehr hoch	Begrenzt	Hoch
Trend 2026	Rückläufig	Steigend	Dominant

1.5 Cloud vs. Edge

1.6 Interoperabilität

Lange Zeit stellten proprietäre Protokolle ein großes Hindernis für die Interoperabilität dar. Heutige Lösungen setzen vor allem auf Matter als zentralen Standard, auf Brücken und Gateways wie Zigbee2MQTT sowie auf Multi-Protokoll-Plattformen wie Home Assistant. Ergänzt werden diese Ansätze durch Thread Border Router. Dennoch bleiben reale Systeme im Jahr 2026 in den meisten Fällen hybrid [Conn24].

2. Abgeleitete Anforderungen

Aus der Recherche ergeben sich folgende zentrale Anforderungen an eine moderne Smart-Home-Infrastruktur:

- **Lose Kopplung:** Komponenten können unabhängig voneinander entwickelt und ersetzt werden. Die Kommunikation erfolgt über definierte Schnittstellen (Pub/Sub, Abschnitt 2.3).
- **Heterogenität:** Geräte, Hersteller und Protokolle lassen sich frei integrieren, ohne dass eine Bindung an ein einzelnes Ökosystem entsteht (Interoperabilität, Abschnitt 2.6).
- **Skalierbarkeit:** Das System bleibt auch bei wachsender Anzahl an Geräten, Nutzern und Nachrichten performant (Pub/Sub + Cloud, Abschnitte 2.3 und 2.5).
- **Verteilung:** Die Verarbeitung erfolgt dezentral und lokal autonom, auch bei Ausfall von Cloud oder Hub (Edge-first, Abschnitt 2.5).
- **Fehlertoleranz:** Teilausfälle führen nicht zum Ausfall des gesamten Systems. Ein resilientes Zustandsmanagement ist vorhanden (Offline-Fähigkeit, Abschnitt 2.5).
- **Erweiterbarkeit:** Neue Geräte, Protokolle und Dienste können hinzugefügt werden, ohne den Kern des Systems verändern zu müssen (Standardisierung, Abschnitt 2.7).

Ergänzend sind Datenschutz durch lokale Verarbeitung, Interoperabilität über einheitliche Standards sowie Wartbarkeit und Beobachtbarkeit durch Logging und klare Schnittstellen zu nennen.

3. Architektur

3.1 Entwurfsprinzip

Kern der Architektur ist ein nachrichtenbasierter Bus auf Basis von MQTT. Alle Teilnehmer kommunizieren ausschließlich über Topics und kennen einander nicht direkt. Dadurch ist lose Kopplung bereits strukturell verankert. Ein zentral definiertes Nachrichtenformat, bestehend aus einem Topic-Schema und einem JSON-Envelope (siehe `common/topics.py`), bildet den verbindlichen Vertrag zwischen allen Komponenten.

3.2 Komponentenübersicht

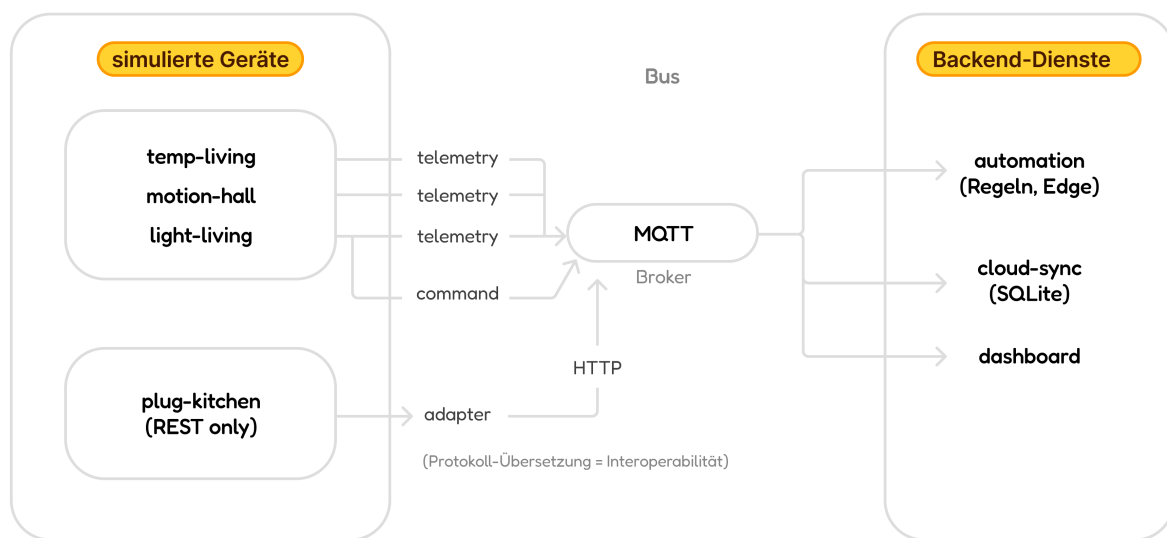


Abbildung 3.1: Komponentenübersicht

3.3 Entwurfsentscheidungen

Folgende zentrale Entwurfsentscheidungen wurden getroffen:

- **Pub/Sub statt Punkt-zu-Punkt:** MQTT dient als zentrales Kommunikationsmodell. Es entkoppelt Sender und Empfänger und ermöglicht das Hinzufügen neuer Teilnehmer, ohne bestehende Komponenten anpassen zu müssen

- **Edge/Cloud-Trennung:** Latenzkritische und lokale Logik, etwa die Regel-Engine, läuft am Edge und bleibt auch ohne Cloud funktionsfähig. Die zentrale Aggregation und Langzeit-Speicherung übernimmt ein separater Dienst (hier SQLite als Cloud-Ersatz).
- **Verzicht auf natives Matter/Thread:** Da physische Geräte nicht im Fokus stehen und eine vollständige Matter-Integration den Rahmen eines Prototyps sprengen würde, dient MQTT als Pub/Sub-Rückgrat. Die Kernidee von Matter (herstellerübergreifende Interoperabilität) wird stattdessen durch das Adapter-Muster umgesetzt. Protokollfremde Geräte werden auf ein einheitliches Nachrichtenformat übersetzt und sind für alle anderen Komponenten nicht mehr von nativen Geräten unterscheidbar.

4. Demonstrator

4.1 Komponenten

Tabelle 4.1: Komponenten des Demonstrators

Komponente	Datei	Rolle
Temperatursensor	<code>devices/temp_sensor.py</code>	Publiziert Telemetrie
Bewegungssensor	<code>devices/motion_sensor.py</code>	Publiziert Ereignisse
Smart-Lampe	<code>devices/smart_light.py</code>	Aktor, abonniert Kommandos
REST-Fremdgerät	<code>rest_device/service.py</code>	HTTP-only Gerät (Heterogenität)
Adapter	<code>rest_device/adapter.py</code>	Brücke zwischen REST und MQTT
Automation-Engine	<code>services/automation.py</code>	Regeln am Edge
Cloud-Sync	<code>services/cloud_sync.py</code>	Aggregation und Historie (SQLite)
Dashboard	<code>services/dashboard.py</code>	Live-Status über HTTP

Jede Komponente läuft als eigener Prozess beziehungsweise in einem eigenen Container (Docker Compose). Damit wird die Anforderung der Verteilung auch auf Betriebssystemebene umgesetzt.

4.2 Schnittstellen

Die Kommunikation erfolgt über einen einheitlichen JSON-Envelope und vier Topic-Klassen. Beispiele sind `home/telemetry/{id}/{metric}` für die Richtung Gerät zum Bus und `home/command/{id}` für die Richtung Bus zum Aktor. Zustand und Verfügbarkeit von Geräten werden über Retained Messages und Last-Will-Nachrichten abgebildet. Die vollständige Definition findet sich in `common/topics.py`.

4.3 Simulation der Geräte

Die simulierten Sensoren erzeugen ihre Werte rein rechnerisch über ein deterministisches Modell. Die Temperatur folgt einer realistischen Tageskurve mit Tiefpunkt in der Nacht und Höchstwert am Nachmittag. Durch eine leichte Trägheit verlaufen aufeinanderfolgende Werte glatt. Die Bewegung folgt einem tageszeitabhängigen Belegungsmuster mit kurzen Aktivitätsphasen. Die Luftfeuchte wird als Random Walk um einen Mittelwert modelliert.

4.4 Anforderungen im Vergleich zur Umsetzung

Die zentralen Anforderungen werden wie folgt umgesetzt:

- **Lose Kopplung:** Die Kommunikation erfolgt über Pub/Sub mit Topics. Ein gemeinsamer Vertrag ist in `common/topics.py` definiert.
- **Heterogenität und Interoperabilität:** Fremdgeräte werden über einen REST-Endpunkt angebunden und durch einen Adapter auf das einheitliche Nachrichtenformat übersetzt.
- **Skalierbarkeit:** Jedes neue Gerät wird als eigener Dienst mit eigenem Topic hinzugefügt, ohne dass der Kern des Systems verändert werden muss.
- **Verteilung:** Jede Komponente läuft als eigener Prozess oder in einem eigenen Container.
- **Fehlertoleranz und Robustheit:** MQTT Last-Will-Nachrichten ermöglichen die Erkennung von Offline-Geräten. Reconnect-Schleifen und das Verwerfen ungültiger Nachrichten sorgen zusätzlich für Stabilität.
- **Erweiterbarkeit:** Eine Sensor-Basisklasse erleichtert die Integration neuer Sensoren. Neue Protokolle können über zusätzliche Adapter eingebunden werden, ohne den Kern zu verändern.

Literaturverzeichnis

- [Conn24] Connectivity Standards Alliance. Build With Matter – Smart Home Device Solution. <https://csa-iot.org/all-solutions/matter/>, 2024.
- [floL26] floLIVE. Edge Computing in 2026: Use Cases, Technology, Edge IoT and Edge AI. <https://fllive.net/blog/glossary/edge-computing-in-2026/>, 2026.
- [Goog24] Google / OpenThread. What is Thread? <https://openthread.io/guides/thread-primer>, 2024.
- [OASI24] OASIS. MQTT – The Standard for IoT Messaging. <https://mqtt.org/>, 2024.
- [Thre24] Thread Group. Thread 1.4 Features White Paper. https://www.threadgroup.org/Portals/0/Documents/Thread_1.4_Features_White_Paper_September_2024.pdf, 9 2024.